PROJECT PROGRESS REPORT: FOCUSLOCK

A Hardware-Free Webcam-Based Gaze-Control System (Full MVP Delivered)

Project Title: FocusLock (Gaze-Controlled Presentation System)	**Delivery Phase:** Working MVP (All 6 Stages Completed)
Domain: Computer Vision, HCI, Signal Processing	**Tech Stack:** Python, OpenCV, MediaPipe, NumPy, PyAutoGUI, Pandas
Current Status: End-to-End Pipeline Verified in Live Testing	**Evaluation Milestone:** Ready for Live Demonstration & Defense

1. Project Objective & Methodology

The objective of **FocusLock** is to develop a contact-free, low-cost presentation navigation system using only a standard laptop webcam. Commercial eye-trackers depend on expensive dedicated infrared cameras. FocusLock achieves real-time gaze tracking through a structured 6-stage software pipeline.

Core Design Methodology: MediaPipe is utilized exclusively as the low-level geometric sensor. The core intellectual and algorithmic contributions of this project are the subsequent stages developed from scratch: **Spatial Normalization**, **Moving-Average Jitter Filtering**, **5-Point Linear Calibration**, **Dwell-Based Intent Recognition**, **OS Actuation Integration**, and **Empirical Telemetry Benchmarking**.

2. Implementation Status Summary (All 6 Stages Completed)

Stage	Module Name	Status	Deliverable Accomplished
Stage 1	Sensing & Iris Extraction	COMPLETED	MediaPipe 478-landmark tracking, iris centroid extraction (468/473).
Stage 2	Gaze Normalization & Filter	COMPLETED	Eye-box ratio calculation, dual-eye fusion, sliding-window jitter filter.
Stage 3	5-Point Screen Calibration	COMPLETED	Interactive 5-target routine, 1D least-squares polynomial fit, JSON caching.
Stage 4	Dwell Intent Detection	COMPLETED	Continuous 1.8s dwell timer state machine eliminating the 'Midas Touch' error.
Stage 5	Presentation Controller	COMPLETED	PyAutoGUI slide-advance keyboard binding with safe DEMO_MODE toggle ('d'
Stage 6	Telemetry Logging & Plots	COMPLETED	Real-time CSV logging & 3-panel matplotlib benchmark analysis script.

3. Detailed Pipeline Architecture & Results

Stage 1: MediaPipe Iris & Landmark Extraction (tracking/iris_tracker.py)

- Configured MediaPipe Tasks FaceLandmarker with standalone 478-point model bundle (indices 468 & 473 for pupil centers), achieving 30+ FPS on CPU.

Stage 2: Gaze Normalization & Moving-Average Filter (tracking/gaze_estimator.py, filtering/moving_average.py)

- Implemented scale-invariant eye-socket boundary ratios: Gaze = (Iris - Eye_min) / Eye_span. Dual-eye fusion averages left and right eye vectors.
- FIFO sliding-window filter (N=7) successfully eliminates high-frequency optical sensor tremor and microsaccades.

Stage 3: 5-Point Calibration & Screen Coordinate Mapping (calibration/calibrator.py)

- 5-point target sequence (Top-L, Top-R, Center, Bottom-L, Bottom-R) maps narrow physiological iris range (0.35-0.65) to screen space [0.0, 1.0] via numpy.polyfit linear regression.

Stage 4: Dwell-Based Intent Detection (intent/dwell_detector.py)

- Formulated continuous 1.8s dwell state machine with single-shot lock, solving the classic 'Midas Touch' false-positive trigger problem.

Stage 5: Presentation Actuation & Safety Toggle (control/presentation_controller.py)

- Dispatches OS keyboard arrow events via PyAutoGUI. Dynamic runtime DEMO_MODE switch ('d' key) allows safe simulation during testing.

Stage 6: Telemetry Logging & Benchmark Graphing (data/logger.py, plot_benchmarks.py)

- Records frame telemetry (timestamp, raw X/Y, filtered X/Y, calibrated X/Y, zone, action) to data/logs/gaze_log.csv. Matplotlib script generates 3-panel scientific benchmark charts.

4. Real-World Limitations & Evaluation Defense

Honest Academic Assessment:

1. **Hardware Boundaries:** Ordinary webcams lack IR illumination; strong ambient lighting shifts can distort eyelid contour detection.
2. **Head Motion:** Pronounced head rotations introduce coordinate drift, solvable in future work via 3D head-pose homography.
3. **Model Simplicity:** Linear least-squares mapping is computationally lightweight and interpretable, though a polynomial model could better capture spherical eyeball geometry.